19 *Case Study: Featherweight Java*

We saw in Chapter 18 how a lambda-calculus with subtyping, records, and references can model certain key features of object-oriented programming. The goal in that chapter was to deepen our understanding of these features by *encoding* them in terms of more elementary ones. In this chapter, we take a different approach, showing how to adapt the ideas in previous chapters to a *direct* treatment of a core object-oriented language based on Java. Prior acquaintance with Java is assumed.

19.1 Introduction

Formal modeling can offer a significant boost to the design of complex real-world artifacts such as programming languages. A formal model may be used to describe some aspect of a design precisely, to state and prove its properties, and to direct attention to issues that might otherwise be overlooked. In formulating a model, however, there is a tension between completeness and compactness: the more aspects the model addresses at the same time, the more unwieldy it becomes. Often it is sensible to choose a model that is less complete but more compact, offering maximum insight for minimum investment. This strategy may be seen in a flurry of recent papers on the formal properties of Java, which omit advanced features such as concurrency and reflection and concentrate on fragments of the full language to which well-understood theory can be applied.

Featherweight Java, or FJ, was proposed by Igarashi, Pierce, and Wadler (1999) as a contender for a *minimal* core calculus for modeling Java's type system. The design of FJ favors compactness over completeness almost obsessively, having just five forms of term: object creation, method invocation,

---

The examples in this chapter are terms of FeatherWeight Java (Figures 19-1 through 19-4). There is no associated OCaml implementation; rather, since FJ is designed to be a strict subset of Java, any Java implementation can be used to run the examples.

field access, casting, and variables. Its syntax, typing rules, and operational semantics fit comfortably on a single (letter-sized) page. Indeed, the aim of its design was to omit as many features as possible—even assignment—while retaining the core features of Java typing. There is a direct correspondence between FJ and a purely functional core of Java, in the sense that every FJ program is literally an executable Java program.

FJ is only a little larger than the lambda-calculus or Abadi and Cardelli's object calculus (1996), and is significantly smaller than other formal models of class-based languages like Java, including those of Drossopoulou, Eisenbach, and Khurshid (1999), Syme (1997), Nipkow and Oheimb (1998), and Flatt, Krishnamurthi, and Felleisen (1998a, 1998b). Being smaller, FJ can focus on just a few key issues. For example, we shall see that capturing the behavior of Java's casting construct in a small-step operational semantics is trickier than we might have expected.

FJ's main application is modeling extensions of Java. Because FJ itself is so compact, it focuses attention on essential aspects of an extension. Moreover, because the proof of type safety for pure FJ is very simple, a rigorous safety proof for even a significant extension may remain manageable. The original FJ paper illustrated this utility by enriching FJ with generic classes and methods *à la* GJ (Bracha, Odersky, Stoutamire, and Wadler, 1998). A followup paper (Igarashi, Pierce, and Wadler, 2001) formalized *raw types,* a feature introduced in GJ to ease evolution of Java programs to GJ. Igarashi and Pierce (2000) used FJ as the basis for a study of Java's *inner class* features. FJ has also been used for studies of type-preserving compilation (League, Trifonov, and Shao, 2001) and semantic foundations (Studer, 2001) for Java.

The goal in designing FJ was to make its proof of type safety as concise as possible, while still capturing the essence of the safety argument for the central features of full Java. Any language feature that made the safety proof *longer* without making it significantly *different* was a candidate for omission. Like other studies of its kind, FJ omits advanced features such as concurrency and reflection. Other Java features missing from FJ include assignment, interfaces, overloading, messages to `super`, `null` pointers, base types (`int`, `bool`, etc.), abstract method declarations, inner classes, shadowing of superclass fields by subclass fields, access control (`public`, `private`, etc.), and exceptions. The features of Java that FJ *does* model include mutually recursive class definitions, object creation, field access, method invocation, method override, method recursion through `this`, subtyping, and casting.

A key simplification in FJ is the omission of assignment. We assume that an object's fields are initialized by its constructor and never changed afterwards. This restricts FJ to a "functional" fragment of Java, in which many common Java idioms, such as use of enumerations, cannot be represented.

Nonetheless, this fragment is computationally complete (it is easy to encode the lambda-calculus into it), and is large enough to include useful programs—for example, many of the programs in Felleisen and Friedman's Java text (1998) use a purely functional style.

19.2 Overview

In FJ, a program consists of a collection of class definitions plus a term to be evaluated, corresponding to the body of the `main` method in full Java. Here are some typical class definitions in FJ.

```
class A extends Object { A() { super(); } }

class B extends Object { B() { super(); } }

class Pair extends Object {
  Object fst;
  Object snd;
  // Constructor:
  Pair(Object fst, Object snd) {
    super(); this.fst=fst; this.snd=snd; }
  // Method definition:
  Pair setfst(Object newfst) {
    return new Pair(newfst, this.snd); } }
```

For the sake of syntactic regularity, we always include the superclass (even when it is `Object`), we always write out the constructor (even for the trivial classes `A` and `B`), and we always name the receiver in a field access or a method invocation (as in `this.snd`), even when the receiver is `this`. Constructors always take the same stylized form: there is one parameter for each field, with the same name as the field; the `super` constructor is invoked to initialize the fields of the superclass; and then the remaining fields are initialized to the corresponding parameters. In this example, the superclass of all three classes is `Object`, which has no fields, so the invocations of `super` have no arguments. Constructors are the only place where `super` or `=` appears in an FJ program. Since FJ provides no side-effecting operations, a method body always consists of `return` followed by a term, as in the body of `setfst()`.

There are five forms of term in FJ. In the example, `new A()`, `new B()`, and `new Pair(...,...)` are *object constructors,* and `... .setfst(...)` is a *method invocation.* In the body of `setfst`, the term `this.snd` is a *field access,* and the occurrences of `newfst` and `this` are *variables.*[1] In the context of the above

1. The syntax of FJ differs slightly from Java in treating `this` as a variable, not a keyword.

definitions, the term

```
new Pair(new A(), new B()).setfst(new B())
```

evaluates to `new Pair(new B(), new B())`.

The remaining form of term is a *cast* (see §15.5). The term

```
((Pair)new Pair(new Pair(new A(), new B()),
                new A()).fst).snd
```

evaluates to `new B()`. The subterm `(Pair)t`, where `t` is `new Pair(...).fst`, is a cast. The cast is required, because `t` is a field access to `fst`, which is declared to contain an `Object`, whereas the next field access, to `snd`, is valid only on a `Pair`. At run time, the evaluation rules check whether the `Object` stored in the `fst` field is a `Pair` (in this case the check succeeds).

Dropping side effects has a pleasant side effect: evaluation can easily be formalized entirely within the syntax of FJ, with no additional mechanisms for modeling the heap (see Chapter 13). There are three basic computation rules: one for field access, one for method invocation, and one for casts. Recall that, in the lambda-calculus, the evaluation rule for applications assumes that the function is first simplified to a lambda abstraction. Similarly, in FJ the evaluation rules assume the object operated upon is first simplified to a `new` term. The slogan in the lambda-calculus is "everything is a function"; here, "everything is an object."

The next example shows the rule for field access (E-ProjNew) in action:

```
new Pair(new A(), new B()).snd   ⟶   new B()
```

Because of the stylized syntax of object constructors, we know that the constructor has one parameter for each field, in the same order that the fields are declared. Here the fields are `fst` and `snd`, and an access to the `snd` field selects the second parameter.

Here is the rule for method invocation (E-InvkNew) in action:

```
new Pair(new A(), new B()).setfst(new B())
⟶  [ newfst ↦ new B(),
     this ↦ new Pair(new A(),new B()) ]
    new Pair(newfst, this.snd)
```

i.e., `new Pair(new B(), new Pair(new A(), new B()).snd)`

The receiver of the invocation is the object `new Pair(new A(),new B())`, so we look up the `setfst` method in the `Pair` class, where we find that it has formal parameter `newfst` and body `new Pair(newfst, this.snd)`. The invocation reduces to the body with the formal parameter replaced by the actual,

and the special variable `this` replaced by the receiver object. This is similar to the beta-reduction rule (E-AppAbs) of the lambda-calculus. The key differences are the fact that the class of the receiver determines where to look for the body (supporting method override), and the substitution of the receiver for `this` (supporting "open recursion through self").[2] In FJ, as in the lambda-calculus, if the formal parameter appears more than once in the body this may lead to duplication of the argument value, but since there are no side effects this difference from the standard Java semantics cannot be observed.

Here is the rule for casts (E-CastNew) in action:

```
(Pair)new Pair(new A(), new B())  ⟶  new Pair(new A(), new B())
```

Once the subject of the cast is reduced to an object, it is easy to check that the class of the constructor is a subclass of the target of the cast. If so (as is the case here), the reduction removes the cast. If not, as in the term `(A)new B()`, then no rule applies and the computation is *stuck,* denoting a run-time error.

There are three ways in which a computation may get stuck: an attempt to access a field not declared for the class, an attempt to invoke a method not declared for the class ("message not understood"), or an attempt to cast to something other than a superclass of an object's run-time class. We will prove that the first two of these never happen in well-typed programs, and that the third never happens in well-typed programs that contain no downcasts (and no "stupid casts"—a technicality explained below).

We adopt a standard call-by-value evaluation strategy. Here are the steps of evaluation for the second example term above, where the next subterm to be reduced is underlined at each step.

```
    ((Pair) (new Pair(new Pair(new A(),new B()), new A())
                .fst).snd
⟶  ((Pair)new Pair(new A(),new B())).snd
⟶  new Pair(new A(), new B()).snd
⟶  new B()
```

19.3 Nominal and Structural Type Systems

Before proceeding with the formal definition of FJ, we should pause to examine one fundamental stylistic difference between FJ (and Java) and the typed lambda-calculi that are the main focus in this book. This difference concerns the status of *type names*.

2. Readers familiar with Abadi and Cardelli's object calculus (1996) will see a strong similarity to their ς-reduction rule.

In previous chapters, we have often defined short names for long or complex compound types to improve the readability of examples, e.g.:

```
NatPair = {fst:Nat, snd:Nat};
```

Such definitions are purely cosmetic: the name `NatPair` is a simple abbreviation for `{fst:Nat,snd:Nat}`, and the two are interchangeable in every context. Our formal presentations of the calculi have ignored abbreviations.

By contrast, in Java, as in many widely used programming languages, type definitions play a much more significant role. Every compound type used in a Java program has a name, and, when we declare the type of a local variable, a field, or a method parameter, we always do so by giving the name. "Bare" types like `{fst:Nat,snd:Nat}` simply cannot appear in these positions.

These type names play a crucial role in Java's subtype relation. Whenever a new name is introduced (in a class or interface definition), the programmer explicitly declares which classes and interfaces the new one extends (or, in the case of a new class and an existing interface, "implements"). The compiler checks these declarations to make sure that the facilities provided by the new class or interface really extend those of each super-class or super-interface—this check corresponds to record subtyping in a typed lambda-calculus. The subtype relation is now defined between type *names* as the reflexive and transitive closure of the declared immediate-subtype relation. If one name has not been declared to be a subtype of another, then it is not.

Type systems like Java's, in which names are significant and subtyping is explicitly declared, are called *nominal*. Type systems like most of the ones in this book, in which names are inessential and subtyping is defined directly on the structures of types are called *structural*.

Nominal type systems have both advantages and disadvantages over structural presentations. Probably the most important advantage is that the type names in nominal systems are useful not only during typechecking, but at run time as well. Most nominal languages tag each run-time object with a header word containing its type name, represented concretely as a pointer to a run-time data structure describing the type and giving pointers to its immediate supertypes. These type tags are handy for a variety of purposes, including run-time type testing (e.g., Java's `instanceOf` test and downcasting operation), printing, marshaling data structures into binary forms for storage in files or transmission over networks, and reflective facilities that permit a program to dynamically investigate the fields and methods of an object that it has been given. Run-time type tags can also be supported in structural systems (see Glew, 1999; League, Shao, and Trifonov, 1999; League, Trifonov, and Shao, 2001; and the citations given there), but they constitute an additional,

separate mechanism; in nominal systems, the run-time tags are identified with the compile-time types.

A less essential, but pleasant, property of nominal systems is that they offer a natural and intuitive account of *recursive types*—types whose definition mentions the type itself. (We will discuss recursive types in detail in Chapter 20.) Such types are ubiquitous in serious programming, being required to describe such common structures as lists and trees, and nominal type systems support them in the most straightforward possible way: referring to `List` in the body of its own declaration is just as easy as referring to any other type. Indeed, even *mutually* recursive types are straightforward. We view the set of type names as being given from the beginning, so that, if the definition of type `A` involves the name `B` and the definition of `B` refers to `A`, there is no issue about "which is defined first." Of course, recursive types can also be handled in structural type systems. Indeed, high-level languages with structural typing, such as ML, generally "bundle" recursive types with other features, so that, for the programmer, they are just as natural and easy to use as in nominal languages. But in calculi intended for more foundational purposes, such as type safety proofs, the mechanisms required to deal rigorously with recursive types can become rather heavy, especially if mutually recursive types are allowed. The fact that recursive types come essentially for free in nominal systems is a decided benefit.

Another advantage of nominal systems is that checking whether one type is a subtype of another becomes almost trivial. Of course, the compiler must still do the work of verifying that the declared subtype relations are safe, which essentially duplicates the structural subtype relation, but this work needs to be done only once per type, at the point where it is defined, rather than during every subtype check. This makes it somewhat easier to achieve good performance in typecheckers for nominal type systems. In more serious compilers, though, it is not clear that the difference between nominal and structural styles has much effect on performance, since well-engineered typecheckers for structural systems incorporate representation techniques that reduce most subtype checks to a single comparison—see page 222.

A final advantage often cited for explicit subtype declarations is that they prevent "spurious subsumption," where the typechecker fails to reject a program that uses a value of one type where a completely different, but structurally compatible, type is expected. This point is more contentious than the ones above, since there are other—arguably better—ways of preventing spurious subsumption, for example using single-constructor datatypes (page 138) or abstract data types (Chapter 24).

Given all these advantages—especially the general usefulness of type tags and the simple treatment of recursive types—it is no surprise to find that

nominal type systems are the norm in mainstream programming languages. The research literature on programming languages, on the other hand, is almost completely concerned with structural type systems.

One immediate reason for this is that, at least in the absence of recursive types, structural systems are somewhat tidier and more elegant. In a structural setting, a type expression is a closed entity: it carries with it all the information that is needed to understand its meaning. In a nominal system, we are always working with respect to some global collection of type names and associated definitions. This tends to make both definitions and proofs more verbose.

A more interesting reason is that the research literature tends to focus on more advanced features—in particular, on powerful mechanisms for type abstraction (parametric polymorphism, abstract data types, user-defined type operators, functors, etc.)—and on languages such as ML and Haskell that embody these features. Unfortunately, such features do not fit very comfortably into nominal systems. A type like `List(T)`, for example, seems irreducibly compound—there will be just one definition somewhere in the program for the constructor `List`, and we need to refer to this definition to see how `List(T)` behaves, so we cannot treat `List(T)` as an atomic name. A few nominal languages have been extended with such "generic" features (Myers, Bank, and Liskov, 1997; Agesen, Freund, and Mitchell, 1997; Bracha, Odersky, Stoutamire, and Wadler, 1998; Cartwright and Steele, 1998; Stroustrup, 1997), but the results of such extensions are no longer pure nominal systems, but somewhat complex hybrids of the two approaches. Designers of languages with advanced typing features thus tend to favor the structural approach.

A full account of the relation between nominal and structural type systems remains a topic of ongoing research.

19.4 Definitions

We now turn to the formal definition of FJ.

Syntax

The syntax of FJ is given in Figure 19-1. The metavariables `A`, `B`, `C`, `D`, and `E` range over class names; `f` and `g` range over field names; `m` ranges over method names; `x` ranges over parameter names; `s` and `t` range over terms; `u` and `v` range over values; `CL` ranges over class declarations; `K` ranges over constructor declarations; `M` ranges over method declarations. We assume that the set of variables includes the special variable `this`, but that `this` is never used as

Syntax

CL	::=	*class declarations:*
		`class C extends C {`$\overline{\texttt{C}}\ \overline{\texttt{f}}$`; K`$\ \overline{\texttt{M}}$`}`
K	::=	*constructor declarations:*
		`C(`$\overline{\texttt{C}}\ \overline{\texttt{f}}$`) {super(`$\overline{\texttt{f}}$`); this.`$\overline{\texttt{f}}$`=`$\overline{\texttt{f}}$`;}`
M	::=	*method declarations:*
		`C m(`$\overline{\texttt{C}}\ \overline{\texttt{x}}$`) {return t;}`
t	::=	*terms:*
		`x` *variable*
		`t.f` *field access*
		`t.m(`$\overline{\texttt{t}}$`)` *method invocation*
		`new C(`$\overline{\texttt{t}}$`)` *object creation*
		`(C) t` *cast*
v	::=	*values:*
		`new C(`$\overline{\texttt{v}}$`)` *object creation*

Subtyping C<:D

$$\texttt{C} <: \texttt{C}$$

$$\frac{\texttt{C} <: \texttt{D} \qquad \texttt{D} <: \texttt{E}}{\texttt{C} <: \texttt{E}}$$

$$\frac{CT(\texttt{C}) = \texttt{class C extends D \{...\}}}{\texttt{C} <: \texttt{D}}$$

Figure 19-1: Featherweight Java (syntax and subtyping)

the name of an argument to a method. Instead, it is considered to be implicitly bound in every method declaration. The evaluation rule for method invocation (rule E-InvkNew in Figure 19-3) will substitute an appropriate object for `this` in addition to substituting argument values for parameters.

We write $\overline{\texttt{f}}$ as shorthand for $\texttt{f}_1,\ldots,\texttt{f}_n$ (similarly $\overline{\texttt{C}}$, $\overline{\texttt{x}}$, $\overline{\texttt{t}}$, etc.) and write $\overline{\texttt{M}}$ for $\texttt{M}_1 \ldots \texttt{M}_n$ (with no commas). We abbreviate operations on pairs of sequences similarly, writing "$\overline{\texttt{C}}\ \overline{\texttt{f}}$" for "$\texttt{C}_1\ \texttt{f}_1,\ldots,\texttt{C}_n\ \texttt{f}_n$", where n is the length of $\overline{\texttt{C}}$ and $\overline{\texttt{f}}$, "$\overline{\texttt{C}}\ \overline{\texttt{f}}$;" for the declarations "$\texttt{C}_1\ \texttt{f}_1;\ldots\texttt{C}_n\ \texttt{f}_n;$" and "`this.`$\overline{\texttt{f}}$`=`$\overline{\texttt{f}}$`;`" for "$\texttt{this.f}_1=\texttt{f}_1;\ldots;\texttt{this.f}_n=\texttt{f}_n;$". Sequences of field declarations, parameter names, and method declarations are assumed to contain no duplicate names.

The declaration `class C extends D {`$\overline{\texttt{C}}\ \overline{\texttt{f}}$`; K`$\ \overline{\texttt{M}}$`}` introduces a class named `C` with superclass `D`. The new class has fields $\overline{\texttt{f}}$ with types $\overline{\texttt{C}}$, a single constructor `K`, and a suite of methods $\overline{\texttt{M}}$. The instance variables declared by `C` are added to the ones declared by `D` and its superclasses, and should have names distinct from these.[3] The methods of `C`, on the other hand, may either over-

3. In Java, instance variables of superclasses may be redeclared—the redeclaration shadows

ride methods with the same names that are already present in D or add new functionality special to C.

The constructor declaration C(D $\overline{\text{g}}$, C $\overline{\text{f}}$) {super($\overline{\text{g}}$); this.$\overline{\text{f}}$=$\overline{\text{f}}$;} shows how to initialize the fields of an instance of C. Its form is completely determined by the instance variable declarations of C and its superclasses: it *must* take exactly as many parameters as there are instance variables, and its body *must* consist of a call to the superclass constructor to initialize its fields from the parameters $\overline{\text{g}}$, followed by an assignment of the parameters $\overline{\text{f}}$ to the new fields of the same names declared by C. (These constraints are enforced by the typing rule for classes in Figure 19-4.) In full Java, a subclass constructor must contain a call to its superclass constructor (when the superclass constructor takes arguments); this is the reason that the constructor body here calls super to initialize the superclass's instance variables. If we did not care about making FJ a literal subset of Java, we could drop the call to super and make each constructor initialize all the instance variables directly.

The method declaration D m(C $\overline{\text{x}}$) {return t;} introduces a method named m with result type D and parameters $\overline{\text{x}}$ of types C. The body of the method is the single statement return t. The variables $\overline{\text{x}}$ are bound in t. The special variable this is also considered bound in t.

A class table *CT* is a mapping from class names C to class declarations CL. A program is a pair (*CT*, t) of a class table and a term. To lighten the notation in what follows, we always assume a *fixed* class table *CT*.

Every class has a superclass, declared with extends. This raises a question: what is the superclass of the Object class? There are various ways to deal with this issue; the simplest one (which we adopt here) is to take Object as a distinguished class name whose definition does *not* appear in the class table. The auxiliary function that looks up fields in the class table is equipped with a special case for Object that returns the empty sequence of fields, denoted •; Object is also assumed to have no methods.[4]

By looking at the class table, we can read off the subtype relation between classes. We write C <: D when C is a subtype of D—i.e., subtyping is the reflexive and transitive closure of the immediate subclass relation given by the extends clauses in *CT*. It is defined formally in Figure 19-1.

The given class table is assumed to satisfy some sanity conditions: (1) $CT(\text{C}) = \text{class C...}$ for every $\text{C} \in dom(CT)$; (2) $\text{Object} \notin dom(CT)$; (3) for every class name C (except Object) appearing anywhere in *CT*, we have $\text{C} \in dom(CT)$; and (4) there are no cycles in the subtype relation induced by *CT*—that is, the <: relation is antisymmetric.

---

the original in the current class and its subclasses. We omit this feature in FJ.

4. In full Java, the class Object actually has several methods. We ignore these in FJ.

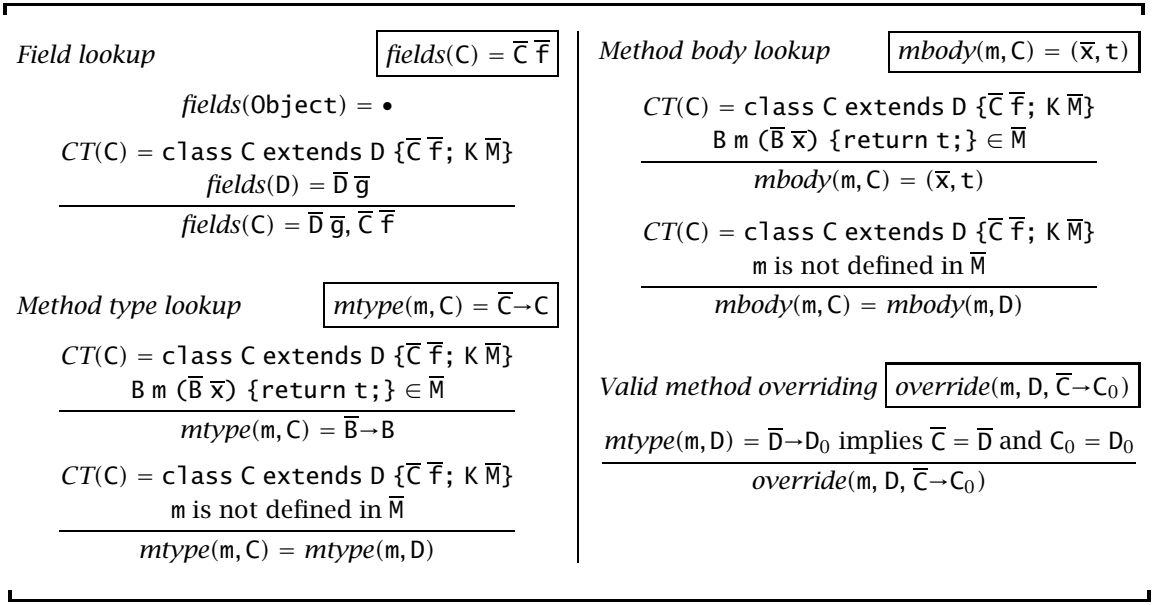


Figure 19-2: Featherweight Java (auxiliary definitions)

Note that the types defined by the class table *are* allowed to be recursive, in the sense that the definition of a class A may use the name A in the types of its methods and instance variables. Mutual recursion between class definitions is also allowed.

19.4.1 EXERCISE [⋆]: By analogy with the S-TOP rule in our lambda-calculus with subtyping, one might expect to see a rule stating that `Object` is a supertype of every class. Why don't we need it here? □

Auxiliary Definitions

For the typing and evaluation rules, we will need a few auxiliary definitions; these are given in Figure 19-2. The fields of a class C, written *fields*(C), is a sequence $\overline{\mathsf{C}}\ \overline{\mathsf{f}}$ pairing the class of each field with its name, for all the fields declared in class C and all of its superclasses. The type of the method m in class C, written *mtype*(m, C), is a pair, written $\overline{\mathsf{B}} \to \mathsf{B}$, of a sequence of argument types $\overline{\mathsf{B}}$ and a single result type B. Similarly, the body of the method m in class C, written *mbody*(m, C), is a pair, written $(\overline{\mathsf{x}}, \mathsf{t})$, of a sequence of parameters $\overline{\mathsf{x}}$ and a term t. The predicate *override*(m, D, $\overline{\mathsf{C}} \to \mathsf{C}_0$) judges whether a method m with argument types $\overline{\mathsf{C}}$ and a result type C_0 may be defined in a subclass of

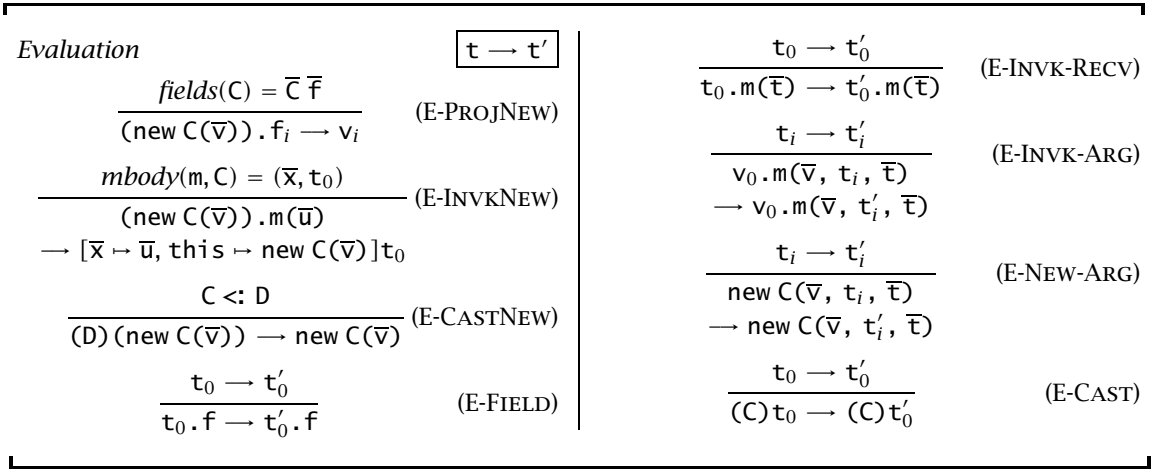


Figure 19-3: Featherweight Java (evaluation)

D. In case of overriding, if a method with the same name is declared in the superclass then it must have the same type.

Evaluation

We use a standard call-by-value operational semantics (Figure 19-3). The three computation rules—for field access, method invocation, and casting—were explained in §19.2. The rest of the rules formalize the call-by-value strategy. The values that can result from normal termination of the evaluator are fully evaluated object creation terms of the form $\texttt{new C}(\overline{\texttt{v}})$.

Note that the run-time behavior of the cast operation is to test whether the actual type of the object being cast is a subtype of the type declared in the cast. If it is, then the cast operation is thrown away and the result is the object itself. This corresponds exactly to the semantics of Java: a run-time cast does not change an object in any way—it simply either succeeds or else fails and raises an exception. In FJ, instead of raising an exception a failing cast will just get stuck, since there will be no evaluation rule that applies.

Typing

The typing rules for terms, method declarations, and class declarations are given in Figure 19-4. An environment Γ is a finite mapping from variables to types, written $\overline{\texttt{x}}:\overline{\texttt{C}}$. Typing statements for terms have the form $\Gamma \vdash \texttt{t} : \texttt{C}$,

Term typing $\boxed{\Gamma \vdash t : C}$

$$\frac{x{:}C \in \Gamma}{\Gamma \vdash x : C} \quad \text{(T-Var)}$$

$$\frac{\Gamma \vdash t_0 : C_0 \qquad \mathit{fields}(C_0) = \overline{C}\ \overline{f}}{\Gamma \vdash t_0.f_i : C_i} \quad \text{(T-Field)}$$

$$\frac{\begin{array}{c}\Gamma \vdash t_0 : C_0 \\ \mathit{mtype}(m, C_0) = \overline{D} \to C \\ \Gamma \vdash \overline{t} : \overline{C} \qquad \overline{C} <: \overline{D}\end{array}}{\Gamma \vdash t_0.m(\overline{t}) : C} \quad \text{(T-Invk)}$$

$$\frac{\begin{array}{c}\mathit{fields}(C) = \overline{D}\ \overline{f} \\ \Gamma \vdash \overline{t} : \overline{C} \qquad \overline{C} <: \overline{D}\end{array}}{\Gamma \vdash \mathsf{new}\ C(\overline{t}) : C} \quad \text{(T-New)}$$

$$\frac{\Gamma \vdash t_0 : D \qquad D <: C}{\Gamma \vdash (C)t_0 : C} \quad \text{(T-UCast)}$$

$$\frac{\Gamma \vdash t_0 : D \qquad C <: D \qquad C \neq D}{\Gamma \vdash (C)t_0 : C} \quad \text{(T-DCast)}$$

$$\frac{\Gamma \vdash t_0 : D \qquad C \not<: D \qquad D \not<: C \qquad \textit{stupid warning}}{\Gamma \vdash (C)t_0 : C} \quad \text{(T-SCast)}$$

Method typing $\boxed{M\ \mathsf{OK\ in}\ C}$

$$\frac{\begin{array}{c}\overline{x} : \overline{C}, \mathsf{this} : C \vdash t_0 : E_0 \qquad E_0 <: C_0 \\ \mathit{CT}(C) = \mathsf{class}\ C\ \mathsf{extends}\ D\ \{\ldots\} \\ \mathit{override}(m, D, \overline{C} \to C_0)\end{array}}{C_0\ m\ (\overline{C}\ \overline{x})\ \{\mathsf{return}\ t_0;\}\ \mathsf{OK\ in}\ C}$$

Class typing $\boxed{C\ \mathsf{OK}}$

$$\frac{\begin{array}{c}K = C(\overline{D}\ \overline{g}, \overline{C}\ \overline{f}) \\ \{\mathsf{super}(\overline{g}); \mathsf{this}.\overline{f} = \overline{f};\} \\ \mathit{fields}(D) = \overline{D}\ \overline{g} \qquad \overline{M}\ \mathsf{OK\ in}\ C\end{array}}{\mathsf{class}\ C\ \mathsf{extends}\ D\ \{\overline{C}\ \overline{f};\ K\ \overline{M}\}\ \mathsf{OK}}$$

Figure 19-4: Featherweight Java (typing)

read "in the environment Γ, term t has type C." The typing rules are syntax directed,[5] with one rule for each form of term, save that there are three rules for casts (discussed below). The typing rules for constructors and method invocations check that each argument has a type that is a subtype of the one declared for the corresponding formal parameter. We abbreviate typing statements on sequences in the obvious way, writing $\Gamma \vdash \overline{t} : \overline{C}$ as shorthand for $\Gamma \vdash t_1 : C_1, \ldots, \Gamma \vdash t_n : C_n$ and $\overline{C} <: \overline{D}$ as shorthand for $C_1 <: D_1, \ldots, C_n <: D_n$.

One minor technical innovation in FJ is the introduction of "stupid" casts. There are three rules for type casts: in an *upcast* the subject is a subclass of the target, in a *downcast* the target is a subclass of the subject, and in a *stupid* cast the target is unrelated to the subject. The Java compiler rejects as ill typed a term containing a stupid cast, but we must allow stupid casts in FJ if we are to formulate type safety as a type preservation theorem for a small-step semantics. This is because a sensible term may be reduced to

5. We follow full Java in choosing an algorithmic formulation of the typing relation. Interestingly, in full Java this choice is actually forced. See Exercise 19.4.6.

one containing a stupid cast. For example, consider the following, which uses classes `A` and `B` as defined in §19.2:

```
(A)(Object)new B() ⟶ (A)new B()
```

We indicate the special nature of stupid casts by including the hypothesis *stupid warning* in the type rule for stupid casts (T-SCAST); an FJ typing corresponds to a legal Java typing only if it does not contain this rule.

Typing statements for method declarations have the form `M OK in C`, read "method declaration `M` is well formed if it occurs in class `C`." It uses the term typing relation on the body of the method, where the free variables are the parameters of the method with their declared types, plus the special variable `this` with type `C`.

Typing statements for class declarations have the form `CL OK`, read "class declaration `CL` is well formed." It checks that the constructor applies `super` to the fields of the superclass and initializes the fields declared in this class, and that each method declaration in the class is ok.

The type of a term may depend on the type of any methods it invokes, and the type of a method depends on the type of a term (its body), so it behooves us to check that there is no ill-defined circularity here. Indeed there is none: the circle is broken because the type of each method is explicitly declared. It is possible to load the class table and use it for typechecking before all the classes in it have been checked, so long as each class is eventually checked.

19.4.2 EXERCISE [⋆⋆]: A number of design decisions in FJ are dictated by the desire to make it a subset of Java, in the sense that every well- or ill-typed FJ program is well- or ill-typed as a Java program, and the well-typed programs behave the same. Suppose this requirement were dropped—i.e., suppose all we wanted was a Java-*like* core calculus. How would you change the design of FJ to make it simpler or more elegant? □

19.4.3 EXERCISE [RECOMMENDED, ⋆⋆⋆ ↛]: The operation of assigning a new value to the field of an object is omitted from FJ to simplify its presentation, but it can be added without changing the basic character of the calculus very much. Do this, using the treatment of references in Chapter 13 as a model. □

19.4.4 EXERCISE [⋆⋆⋆ ↛]: Extend FJ with analogs of Java's `raise` and `try` forms, using the treatment of exceptions in Chapter 14 as a model. □

19.4.5 EXERCISE [⋆⋆ ↛]: FJ, like full Java, presents the typing relation in algorithmic form. There is no subsumption rule; instead, several of the other rules include subtyping checks among their premises. Can the system be reformulated in a more declarative style, dropping most or all of these premises in favor of a single subsumption rule? □

19.4.6 EXERCISE [⋆⋆⋆]: Full Java provides both classes and *interfaces,* which specify the types of methods, but not their implementations. Interfaces are useful because they permit a richer, non-tree-structured, subtype relation: each class has a single superclass (from which it inherits instance variables and method bodies), but may additionally implement any number of interfaces.

The presence of interfaces in Java actually forces the choice of an algorithmic presentation of the typing relation, which gives each typable term a unique (minimal) type. The reason is an interaction between conditional expressions (written $\mathtt{t_1\ ?\ t_2\ :\ t_3}$ in Java) and interfaces.

1. Show how to extend FJ with interfaces in the style of Java.
2. Show that, in the presence of interfaces, the subtype relation is not necessarily closed under joins. (Recall from §16.3 that the existence of joins played a critical role in the minimal typing property for conditionals.)
3. What is Java's typing rule for conditional expressions? Is it reasonable? □

19.4.7 EXERCISE [⋆⋆⋆]: FJ includes Java's `this` keyword, but omits `super`. Show how to add it. □

19.5 Properties

We can prove a standard type-preservation theorem for FJ.

19.5.1 THEOREM [PRESERVATION]: If $\Gamma \vdash \mathtt{t} : \mathtt{C}$ and $\mathtt{t} \longrightarrow \mathtt{t'}$, then $\Gamma \vdash \mathtt{t'} : \mathtt{C'}$ for some $\mathtt{C'} <: \mathtt{C}$. □

Proof: Exercise [⋆⋆⋆]. □

We can also show a variant of the standard progress theorem: if a program is well typed, then the only way it can get stuck is if it reaches a point where it cannot perform a downcast. We use the mechanism of *evaluation contexts* to identify the failing downcast in the latter case.

19.5.2 LEMMA: Suppose $\mathtt{t}$ is a well-typed term.

1. If $\mathtt{t} = \mathtt{new\ C_0(\overline{t}).f}$, then $\mathit{fields}(\mathtt{C_0}) = \overline{\mathtt{C}}\ \overline{\mathtt{f}}$ and $\mathtt{f} \in \overline{\mathtt{f}}$.
2. If $\mathtt{t} = \mathtt{new\ C_0(\overline{t}).m(\overline{s})}$, then $\mathit{mbody}(\mathtt{m}, \mathtt{C_0}) = (\overline{\mathtt{x}}, \mathtt{t_0})$ and $|\overline{\mathtt{x}}| = |\overline{\mathtt{s}}|$. □

Proof: Straightforward. □

19.5.3 DEFINITION: The set of *evaluation contexts* for FJ is defined as follows:

E	::=	*evaluation contexts:*
	$[\,]$	*hole*
	$E.\mathtt{f}$	*field access*
	$E.\mathtt{m}(\overline{\mathtt{t}})$	*method invocation (receiver)*
	$\mathtt{v}.\mathtt{m}(\overline{\mathtt{v}},E,\overline{\mathtt{t}})$	*method invocation (arg)*
	$\mathtt{new}\ \mathtt{C}(\overline{\mathtt{v}},E,\overline{\mathtt{t}})$	*object creation (arg)*
	$(\mathtt{C})E$	*cast*

Each evaluation context is a term with a hole (written [ ]) somewhere inside it. We write $E[\mathtt{t}]$ for the ordinary term obtained by replacing the hole in E with $\mathtt{t}$. □

Evaluation contexts capture the notion of the "next subterm to be reduced," in the sense that, if $\mathtt{t} \longrightarrow \mathtt{t}'$, then we can express $\mathtt{t}$ and $\mathtt{t}'$ as $\mathtt{t} = E[\mathtt{r}]$ and $\mathtt{t}' = E[\mathtt{r}']$ for a unique E, $\mathtt{r}$, and $\mathtt{r}'$, with $\mathtt{r} \longrightarrow \mathtt{r}'$ by one of the computation rules E-PROJNEW, E-INVKNEW, or E-CASTNEW.

19.5.4 THEOREM [PROGRESS]: Suppose $\mathtt{t}$ is a closed, well-typed normal form. Then either (1) $\mathtt{t}$ is a value, or (2) for some evaluation context E, we can express $\mathtt{t}$ as $\mathtt{t} = E[(\mathtt{C})(\mathtt{new}\ \mathtt{D}(\overline{\mathtt{v}}))]$, with $\mathtt{D} \not<: \mathtt{C}$. □

Proof: Straightforward induction on typing derivations. □

The progress property can be sharpened a little: if $\mathtt{t}$ contains only upcasts, then it cannot get stuck (and, if the original program contains only upcasts, then evaluation will never produce any casts that are not upcasts). But, in general, we want to use casting to *lower* the static types of objects, and so we must live with the possibility that casts can fail at run time. In full Java, of course, a cast failure does not stop the whole program: it generates an exception that can be caught by a surrounding exception handler.

19.5.5 EXERCISE [⋆⋆⋆ ↛]: Starting from one of the lambda-calculus typecheckers, build a typechecker and interpreter for Featherweight Java. □

19.5.6 EXERCISE [⋆⋆⋆⋆ ↛]: The original FJ paper (Igarashi, Pierce, and Wadler, 1999) also formalizes polymorphic types in the style of GJ. Extend the typechecker and interpreter from Exercise 19.5.5 to include these features. (You will need to read Chapters 23, 25, 26, and 28 before attempting this exercise.) □

19.6 Encodings vs. Primitive Objects

We have seen two contrasting approaches to the semantics and typing of simple object-oriented languages. In Chapter 18, we used combinations of

features from the simply typed lambda-calculus with records, references, and subtyping to encode objects, classes, and inheritance. In the present chapter, we gave a direct account of a simple language in which objects and classes are primitive mechanisms.

Each of these approaches has its uses. Studying object encodings lays bare the fundamental mechanisms of encapsulation and reuse and allows us to compare them to other mechanisms with related aims. These encodings also help in understanding the way objects are translated into yet lower-level languages by compilers, and in understanding the interactions between objects and other language features. Treating objects as primitive, on the other hand, allows us to talk about their operational semantics and typing behavior directly; this kind of presentation is a better tool for high-level language design and documentation.

Ultimately, of course, we would like to have both views—a high-level language including primitive features for objects, classes, etc., with its own typing rules and operational semantics, plus a translation from this language to some lower-level language with just records and functions (or, indeed, an even lower-level language with nothing but registers, pointers, and instruction sequences), and finally a proof that this translation is a correct implementation of the high-level language, in the sense that the translation preserves the evaluation and typing properties of the high-level language. Many variants of this exercise have been carried out—for FJ itself by League, Trifonov, and Shao (2001), and for other object-oriented core calculi by Hofmann and Pierce (1995b), Bruce (1994, 2002), Abadi, Cardelli, and Viswanathan (1996), and others.

19.7 Notes

This chapter is adapted from the original FJ article by Igarashi, Pierce, and Wadler (1999). The main difference in presentation is that we have used a call-by-value operational semantics, for consistency with the rest of the book, while the original used a nondeterministic beta-reduction relation.

There have been several proofs of type safety for subsets of Java. In the earliest, Drossopoulou, Eisenbach, and Khurshid (1999), using a technique later mechanically checked by Syme (1997), prove safety for a substantial subset of sequential Java. Like FJ, they use a small-step operational semantics, but they avoid the subtleties of "stupid casts" by omitting casting entirely. Nipkow and Oheimb (1998) give a mechanically checked proof of safety for a somewhat larger core language. Their language does include casts, but it is formulated using a "big-step" operational semantics, which sidesteps the

stupid cast problem. Flatt, Krishnamurthi, and Felleisen (1998a, 1998b) use a small-step semantics and formalize a language with both assignment and casting, treating stupid casts as in FJ. Their system is somewhat larger than FJ (the syntax, typing, and operational semantics rules take perhaps three times the space), and its safety proof, though correspondingly longer, is of similar complexity.

Of these three studies, Flatt, Krishnamurthi, and Felleisen's is closest to FJ in an important sense: the goal there, as here, is to choose a core calculus that is as *small* as possible, capturing just the features of Java that are relevant to some particular task. In their case, the task is analyzing an extension of Java with Common Lisp style mixins. The goal of the other two systems mentioned above, on the other hand, is to include as *large* a subset of Java as possible, since their primary interest is proving the safety of Java itself.

The literature on foundations of object-oriented languages contains many papers on formalizing class-based object-oriented languages, either taking classes as primitive (e.g., Wand, 1989a, Bruce, 1994, Bono, Patel, Shmatikov, and Mitchell, 1999b, Bono, Patel, and Shmatikov, 1999a) or translating classes into lower-level mechanisms (e.g., Fisher and Mitchell, 1998, Bono and Fisher, 1998, Abadi and Cardelli, 1996, Pierce and Turner, 1994).

A related thread of work considers models of object-oriented languages in which classes are replaced by some form of *method override* or *delegation* (Ungar and Smith, 1987), where individual objects may inherit behaviors from other *objects*. The resulting calculi tend to be somewhat simpler than those for class-based languages, since they deal with a smaller set of concepts. The most highly developed and best known of these is Abadi and Cardelli's *object calculus* (1996). Another popular one was developed by Fisher, Honsell, and Mitchell (1994).

A rather different approach to objects, classes, and inheritance, known as *multi-methods,* has been formalized by Castagna, Ghelli, and Longo (1995). The footnote on page 226 gives additional citations.

Inside every large language is a small language struggling to get out....
—Igarashi, Pierce, and Wadler (1999)

Inside every large program is a small program struggling to get out....
—Tony Hoare, Efficient Production of Large Programs *(1970)*

I'm fat, but I'm thin inside.
Has it ever struck you that there's a thin man inside every fat man?
—George Orwell, Coming Up For Air *(1939)*